上海大学
SHANGHAI UNIVERSITY

本科毕业论文（设计）

UNDERGRADUATE THESIS (PROJECT)

题　　目：基于嵌入式平台的目标检测系统研究

学　　院：通信与信息工程学院

专　　业：电子信息工程

学　　号：22123739

学生姓名：朱奕澄

指导教师：滕国伟

起讫日期：

2025.12.29-2026.05.22

姓　　名：　　学号：

论文题目：

原 创 性 声 明

本人声明：所呈交的论文是本人在指导教师指导下进行的研究工作。除了文中特别加以标注和致谢的地方外，论文中不包含其他人已发表或撰写过的研究成果。参与同一工作的其他同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

签 名：_____日 期：_____

本论文使用授权说明

本人完全了解上海大学有关保留、使用学位论文的规定，即：学校有权保留论文及送交论文复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容。

（**保密的论文在解密后应遵守此规定**）

签 名：_____指导教师签名：_____日期：_____

(注：本页由学生和老师手写签名，然后以扫描页插入论文 PDF 版)

摘要

随着无人机在低空安防、电力巡检和城市管理等场景中的应用增多，面向边缘端的实时目标检测系统具有重要的工程应用价值。针对传统云端检测方案存在带宽占用高、端到端延迟大和现场部署灵活性不足等问题，本文围绕 RK3588 嵌入式平台设计并实现了一套无人机目标检测系统。系统以 YOLOv10 单类无人机检测模型为基础，完成 PyTorch、ONNX 到 RKNN 模型的迁移与板端适配，并采用 C/C++ 实现固定视频检测和实时 RTSP 检测程序。

在系统实现方面，本文构建了视频输入、图像预处理、NPU 推理、后处理、显示推流和日志记录相互解耦的处理流程。针对实时检测过程中容易出现的吞吐不足、端到端延迟升高和检测框抖动问题，系统实现了检测间隔控制、动态 ROI、轻量跟踪、检测框平滑和多 context NPU 并行推理等策略。实验结果表明，恢复后的单类无人机 RKNN 模型能够在 RK3588 上完成固定视频检测，并可通过 RTSP 输出带框视频流。多 context 机制在每帧检测场景下能够提高 NPU 推理吞吐，而检测间隔与框平滑策略更适合低延迟实时观看场景。本文工作为目标检测算法在国产嵌入式平台上的工程化部署与系统级优化提供了参考。

关键词：嵌入式平台；目标检测；RK3588；神经网络处理单元；多线程

ABSTRACT

With the increasing use of unmanned aerial vehicles in low-altitude security, power inspection, and urban management, real-time object detection on edge devices has significant engineering value. To address the high bandwidth consumption, large end-to-end latency, and limited deployment flexibility of cloud-based detection schemes, this thesis designs and implements a UAV object detection system based on the RK3588 embedded platform. The system uses a single-class YOLOv10 UAV detection model as the baseline, completes the model migration from PyTorch and ONNX to RKNN, and implements both fixed-video detection and real-time RTSP detection programs in C/C++.

In terms of system implementation, the proposed system decouples video input, image preprocessing, NPU inference, post-processing, display streaming, and logging. To reduce throughput bottlenecks, end-to-end latency, and bounding-box jitter in real-time detection, the system implements detection interval control, dynamic ROI, lightweight tracking, bounding-box smoothing, and multi-context NPU parallel inference. Experimental results show that the recovered single-class UAV RKNN model can perform fixed-video detection on RK3588 and output annotated video streams through RTSP. The multi-context mechanism improves inference throughput in frame-by-frame detection, while detection interval control and bounding-box smoothing are more suitable for low-latency real-time viewing. This work provides a practical reference for engineering deployment and system-level optimization of object detection algorithms on domestic embedded platforms.

Keywords: Embedded platform; Object detection; RK3588; Neural Processing Unit; Multithreading

目　录

1.1 研究背景与意义 6
1.2 国内外研究现状 6
1.3 本文主要研究内容 7
1.4 论文组织结构 7
第二章 相关技术基础 9
2.1 目标检测算法基础 9
2.2 无人机小目标检测特点 9
2.3 RK3588 嵌入式平台 9
2.4 RKNN 模型部署流程 10
2.5 C/C++ 多线程与实时视频流水线 10
2.6 本章小结 11
第三章 系统总体设计 12
3.1 系统需求分析 12
3.1.1 功能需求 12
3.1.2 性能需求 13
3.1.3 部署约束 14
3.2 系统总体架构 14
3.3 模型移植与检测模块设计 15
3.3.1 模型移植流程 15
3.3.2 检测结果解析 16
3.4 实时视频处理流水线设计 17
3.4.1 多线程流水线结构 17
3.4.2 实时输出方式 17
3.5 动态 ROI 与轻量跟踪策略设计 17
3.5.1 检测间隔策略 17
3.5.2 动态 ROI 策略 18
3.5.3 轻量跟踪与框平滑 18
3.6 多 context NPU 并行推理设计 19

3.6.1 多 context 设计思想 19
3.6.2 多 context 与检测间隔的关系 19
3.6.3 阶段耗时分析设计 20
3.7 本章小结 20
第四章 系统实现与调试 21
4.1 开发与部署环境 21
4.2 模型导出与 RKNN 转换实现 23
4.3 RKNN 推理模块实现 24
4.4 固定视频检测程序实现 25
4.5 实时 RTSP 检测程序实现 26
4.6 检测间隔、动态 ROI 与轻量跟踪实现 28
4.7 多 context NPU 并行推理实现 30
4.8 日志记录与性能调试接口 32
4.9 本章小结 33
第五章 系统测试与实验分析 34
5.1 实验环境与评价指标 34
5.2 无人机检测模型训练结果 36
5.3 公开视频固定输入验证 37
5.4 实时 RTSP 首轮配置对比 38
5.5 检测间隔与框平滑策略实验 39
5.6 多 context NPU 并行推理实验 41
5.7 阶段耗时与瓶颈分析 42
5.8 Zero-copy 输入路径探索 44
5.9 综合讨论与推荐配置 45
5.10 本章小结 46
总结与展望 48
参考文献 49
致谢 50

1.1 研究背景与意义

近年来，无人机在低空巡检、公共安全、应急救援和城市管理等领域得到广泛应用。无人机具有机动性强、部署灵活和使用成本较低等特点，但在复杂低空环境中也带来了目标监测、区域安全和实时预警等方面的新需求。对于无人机检测任务，如果将视频流传输到云端服务器进行处理，系统往往需要占用较高网络带宽，并引入额外传输延迟。当应用场景需要在本地快速完成目标识别和响应时，云端处理方式难以满足实时性和稳定性要求。

边缘计算为上述问题提供了可行路径。将目标检测模型部署到嵌入式平台后，系统可以在靠近数据源的位置完成视频采集、图像预处理、模型推理和结果输出，从而降低网络依赖并提升响应速度。RK3588 平台集成多核 CPU、GPU 和神经网络处理单元（Neural Processing Unit，NPU），具备较强的端侧推理能力，适合用于构建实时目标检测系统。然而，在实际部署中，模型格式转换、算子兼容、输入输出张量解析、视频流处理、多线程调度和硬件资源利用之间存在紧密耦合，单纯完成模型转换并不能保证系统达到实时运行效果。

因此，本文以“基于嵌入式平台的目标检测系统研究”为题，围绕无人机识别算法在 RK3588 平台上的工程化移植与实时部署展开研究。本文重点不在于提出新的目标检测网络结构，而是在现有检测模型基础上完成面向嵌入式平台的模型迁移、C/C++部署、多线程流水线设计、NPU 多 context 推理和系统性能分析。该研究对于理解目标检测算法从训练环境走向边缘设备的完整工程链路具有一定实践意义。

1.2 国内外研究现状

目标检测是计算机视觉领域的重要任务之一，其目标是在图像或视频中定位并识别特定类别目标。以 YOLO 系列为代表的一阶段检测算法具有结构简洁、推理速度快和工程部署方便等特点，在实时检测任务中应用广泛[1-3]。随着网络结构、特征融合和训练策略不断改进，目标检测算法在通用数据集上的精度和速度均有明显提升。

无人机检测属于小目标检测的典型应用场景。远距离无人机在图像中往往只占据少量像素，且容易受到天空、建筑、树枝、鸟类和光照变化等干扰。针对这类任务，

研究者通常从高分辨率特征、注意力机制、跨尺度融合、样本增强和专用数据集构建等方面提升检测能力[4-5]。导师提供的 SDD-YOLO 和 UAV-DETR 相关资料也说明，面向反无人机场景的算法研究正逐步关注小目标表征、轻量化结构和边缘部署友好性。

在嵌入式部署方面，研究重点从单纯模型压缩逐步转向系统级协同优化[6-7]。实际运行帧率不仅取决于神经网络推理耗时，还受到图像预处理、数据拷贝、后处理、编码推流、线程同步和队列积压等因素影响。对于 RK3588 这类异构平台，合理使用 NPU、CPU 和视频处理组件，是提升系统整体性能的重要手段。因此，端侧目标检测系统需要在模型迁移、运行时接口和多线程调度之间进行综合设计。

1.3 本文主要研究内容

本文围绕 RK3588 嵌入式平台上的无人机目标检测系统开展研究，主要内容包括以下几个方面。

1. 完成无人机检测模型从训练环境到 RK3588 平台的迁移。本文以已训练的 YOLOv10 单类无人机检测模型为基础，完成 PyTorch 权重、ONNX 模型和 RKNN 模型之间的转换与验证，并对板端输出张量进行解析。

2. 设计并实现 C/C++ 固定视频检测和实时 RTSP 检测程序。固定视频程序用于可重复实验和公开视频验证，实时 RTSP 程序用于 USB 摄像头或本地视频输入下的实时显示。

3. 构建面向实时检测的多线程处理流程。系统将视频读取、NPU 推理、结果绘制、RTSP 推流和日志记录进行解耦，降低单一串行流程对实时性的影响。

4. 实现检测间隔、动态 ROI、轻量跟踪和框平滑策略。上述策略用于降低不必要的 NPU 调用，并改善实时观看时检测框抖动和延迟问题。

5. 开展 RK3588 NPU 多 context 并行推理实验。本文比较单 context、双 context 和多 context 配置下的运行表现，并结合阶段耗时分析定位系统瓶颈。

1.4 论文组织结构

本文共分为五章。第一章介绍研究背景、研究意义、国内外研究现状和本文主要研究内容。第二章介绍目标检测、无人机小目标检测、RK3588 平台、RKNN 部署和 C/C++ 多线程视频处理等相关技术基础。第三章给出系统总体设计，包括需求分析、

总体架构、模型迁移设计、实时视频处理流水线、动态 ROI 与多 context 推理设计。第四章阐述系统实现与调试过程，包括开发环境、模型转换、RKNN 推理模块、固定视频程序、实时 RTSP 程序和日志接口。第五章给出系统测试与实验分析，包括训练结果、公开视频固定输入验证、实时 RTSP 实验、多 context 对比、阶段耗时分析和 zero-copy 输入实验。

第二章 相关技术基础

2.1 目标检测算法基础

目标检测任务需要同时完成目标类别判断和边界框定位。与图像分类任务相比，目标检测不仅需要判断图像中是否存在某类目标，还需要给出目标在图像中的空间位置。现代目标检测算法通常由主干网络、特征融合模块和检测头组成。主干网络负责提取图像特征，特征融合模块用于整合不同尺度信息，检测头输出目标框位置、类别和置信度。

YOLO 系列算法属于一阶段目标检测方法，其基本思想是在一次前向推理中直接预测目标类别和边界框位置[1-3]。该类算法具有推理速度快、部署链路成熟和工程生态完善等优点，因此常被用于实时检测场景。本文所使用的无人机检测模型属于单类别目标检测模型，板端后处理重点集中在边界框解析、置信度筛选、非极大值抑制和坐标还原等步骤。

2.2 无人机小目标检测特点

无人机检测与通用目标检测相比具有更明显的小目标特征[4-5]。在地面对空拍摄场景中，远距离无人机在画面中所占像素较少，目标外观信息有限，且容易与鸟类、云层、树枝、建筑边缘和图像噪声混淆。当目标尺寸较小时，轻微的定位偏差就可能导致 IoU 指标明显下降，从而影响检测评价结果。

此外，无人机检测系统通常具有较强实时性要求。对于固定视频实验，系统需要能够稳定输出逐帧检测结果和可复现实验日志；对于实时演示场景，系统还需要保证输出画面连续、检测框稳定、延迟较低。上述特点决定了本文不仅需要关注模型检测精度，还需要关注端侧系统吞吐、延迟、资源占用和显示稳定性。

2.3 RK3588 嵌入式平台

RK3588 是面向边缘计算场景的高性能嵌入式处理平台，集成多核 CPU、GPU、视频编解码模块和 NPU 等异构计算资源。对于目标检测任务，NPU 可承担神经网络前向推理计算，CPU 则负责视频输入、数据组织、后处理和线程调度。与通用 PC 或

服务器环境相比，嵌入式平台在功耗、内存带宽、接口能力和运行稳定性方面具有不同约束。

在实际部署过程中，系统性能并不只由 NPU 推理时间决定。图像从摄像头或视频文件进入系统后，需要经过解码、缩放、颜色空间转换、输入张量设置、模型推理、输出解析、绘制和推流等步骤。任一环节耗时过高，都可能造成队列积压和端到端延迟升高。因此，在 RK3588 上实现实时检测，需要从系统级角度组织各处理模块。

2.4 RKNN 模型部署流程

RKNN 是 Rockchip 面向其 NPU 平台提供的模型格式和运行时接口。通常情况下，训练得到的 PyTorch 权重需要先导出为 ONNX 模型，再通过 RKNN Toolkit2 转换为 RKNN 模型文件[7,9]。转换过程中需要关注输入尺寸、算子兼容性、模型输出格式和量化配置等问题。若模型包含不受支持的算子，可能导致转换失败或运行时性能下降。

本文当前稳定主路径采用浮点 RKNN 模型。虽然 INT8 量化理论上可以降低计算和存储开销，但量化需要校准数据、误差分析和板端精度验证。考虑到当前系统已稳定跑通浮点模型，本文将 FP RKNN 作为主要实验对象，INT8 量化作为后续优化方向进行讨论，而不将其写成已经完整完成的优化结果。

2.5 C/C++ 多线程与实时视频流水线

实时视频检测系统通常包含多个具有不同耗时特征的处理阶段。如果所有阶段在单线程中串行执行，系统整体帧率会受到最慢阶段限制，同时难以充分利用多核 CPU 和 NPU 资源。多线程流水线通过将采集、推理、发布等模块解耦，可以在不同帧之间形成重叠执行，从而提升吞吐能力。

本文系统采用 C/C++ 实现板端程序，主要原因在于 C/C++ 便于直接调用 RKNN Runtime、OpenCV 和 GStreamer 等底层库，并能够更精细地控制线程、队列和内存生命周期[7,10-11]。在线程协作方面，系统使用有限长度队列连接不同模块，避免输入速度高于处理速度时造成无限堆积。对于实时显示任务，系统更关注最新帧和低延迟，因此在必要时允许丢弃过旧帧，以换取更好的观看响应。

2.6 本章小结

本章介绍了本文涉及的目标检测、无人机小目标检测、RK3588 平台、RKNN 部署和 C/C++ 多线程视频处理等基础内容。上述技术构成了后续系统总体设计、板端实现和性能实验分析的基础。

第三章 系统总体设计

本章围绕“基于嵌入式平台的目标检测系统研究”的总体实现目标，对无人机目标检测系统的功能需求、性能需求、部署约束和模块架构进行设计。系统以 RK3588 嵌入式平台为主要部署对象，利用板载神经网络处理单元（NPU）完成目标检测模型推理，并结合实时视频采集、检测结果后处理、轻量级目标跟踪、动态感兴趣区域（ROI）以及多推理上下文（context）并行推理策略，实现面向边缘端场景的目标检测与显示。

与单纯在 PC 端运行目标检测算法不同，嵌入式部署需要同时关注检测精度、实时性、资源占用和系统稳定性。本文的系统设计并不追求在所有公开数据集上获得最高精度，而是以“模型能够在 RK3588 平台稳定运行，并在实时视频流中输出可观察、可记录、可复现实验结果”为核心目标。因此，本章将系统划分为模型移植、视频输入、NPU 推理、后处理与显示、性能记录和多线程优化等模块，并为后续系统实现与实验分析提供设计依据。

3.1 系统需求分析

3.1.1 功能需求

本文系统面向嵌入式平台上的小目标检测应用，主要完成视频输入、目标检测、结果可视化和实验记录等功能。系统需要能够读取本地视频、USB 摄像头或实时流输入，在每帧图像中识别目标位置，并将检测框、类别名称和置信度绘制到输出画面中。为了支撑论文实验，系统还需要保存检测结果、运行帧率、推理耗时和资源占用等数据，使同一段视频能够在不同配置下进行重复测试。

系统的主要功能需求如表 3.1 所示。

表 3.1 系统功能需求

序号	功能模块	需求说明
1	视频输入	支持本地视频文件、USB 摄像头和实时视频流输入，并能够在固定视频上进行重复实验。
2	模型推理	在 RK3588 平台上加载 RKNN 模型，调用 NPU 完成目标检测推理。
3	目标后处理	对模型输出进行解析、置信度筛选和非极大值抑制，得到目标框坐标和置信度。
4	可视化显示	在输出画面中绘制检测框、类别名称和置信度，并通过视频文件或 RTSP 流显示检测结果。
5	性能记录	记录流处理帧率、NPU 推理帧率、端到端延迟、CPU 占用和内存占用等指标。
6	策略配置	支持检测间隔、多线程推理、平滑参数、ROI 策略等运行参数配置，便于实验对比。

3.1.2 性能需求

嵌入式目标检测系统需要在有限算力和功耗条件下保持较高实时性。由于无人机等小目标在画面中可能只占据较少像素，系统既要尽量保证检测召回率，又要避免过高的推理频率导致视频处理阻塞。结合 RK3588 平台的实际部署情况，本文将性能需求分为实时性、稳定性、可复现性和资源约束四个方面。

首先，系统需要保证视频流能够连续输出，避免因推理过慢造成明显卡顿。其次，检测结果需要具有一定稳定性，避免检测框在相邻帧之间出现明显抖动。再次，系统需要能够在相同输入视频和相同参数下得到可复现的实验日志。最后，系统应尽量降低 CPU 端后处理和视频编码的负担，将主要神经网络计算交由 NPU 完成。

表 3.2 给出了本文系统设计中关注的主要性能指标。

表 3.2 系统性能指标

指标名称	含义	设计目标
stream_fps	输出视频流的实际帧率	保证画面连续，尽量接近输入视频帧率。
npu_fps	NPU 完成推理的帧率	反映模型推理吞吐能力，用于比较不同 context 数量。
端到端延迟	输入帧进入系统到输出检测结果的整体延迟（end-to-end latency）	在实时显示场景下尽量降低。
CPU 占用率	视频读取、预处理、后处理和编码造成的 CPU 负载	避免 CPU 成为主要瓶颈。
RSS 内存	程序运行时常驻内存占用	保证长时间运行稳定。
误检与漏检	检测结果中的错误目标和漏掉目标	用于分析模型在公开视频和真实输入中的适用性。

3.1.3 部署约束

本文系统的部署对象为 RK3588 嵌入式平台。该平台具备多核 CPU、GPU 和 NPU 等计算资源，但在实际系统中，视频采集、图像预处理、NPU 推理、后处理和视频输出之间仍然存在明显的调度关系。如果各模块串行执行，任一环节耗时过长都会影响实时显示效果。因此，系统需要采用多线程流水线结构，将视频采集、模型推理和结果输出进行解耦。

此外，模型格式也是部署约束之一。训练阶段通常得到 PyTorch 权重文件或 ONNX 模型，而 RK3588 的 NPU 推理需要使用 RKNN 格式模型[7,9]。因此，系统需要建立从训练模型到 RKNN 模型的转换流程，并在板端验证模型输入输出张量是否与后处理代码匹配。本文当前稳定运行的主路径采用浮点 RKNN 模型，INT8 量化优化和 RGA 硬件预处理作为后续扩展方向，不在本章中作为已完成主线进行描述。

3.2 系统总体架构

本文系统采用“视频输入层、预处理层、NPU 推理层、后处理层、策略控制层和

结果输出层”的分层架构，如图 3.1 所示。视频输入层负责从本地文件、摄像头或 RTSP 输入中获取图像帧；预处理层负责完成图像缩放、颜色空间转换和输入张量组织；NPU 推理层负责调用 RKNN Runtime 完成神经网络前向计算；后处理层负责解析模型输出并生成检测框；策略控制层负责检测间隔、结果复用、动态 ROI、框平滑和轻量跟踪；结果输出层负责视频显示、RTSP 推流和日志保存。

图 3.1 系统总体架构图

在系统运行过程中，视频帧首先进入采集线程。采集线程不直接等待 NPU 推理完成，而是将帧数据写入有限长度队列。推理线程从队列中取出图像帧，按照当前策略决定是否执行完整 NPU 检测。如果当前帧需要检测，则调用 RKNN 模型并更新检测结果；如果当前帧不需要检测，则根据上一帧检测框和轻量跟踪结果进行复用。输出线程负责将最新检测结果绘制到画面中，并通过视频文件或 RTSP 服务输出。

这种设计将视频输入、模型推理和显示输出解耦，避免单个慢速模块阻塞整个系统。对于固定视频实验，系统可以保存带框输出视频和逐帧检测结果；对于实时演示，系统可以通过 RTSP 地址在电脑端查看检测画面。固定视频实验保证可复现性，实时视频流验证实际部署效果，两者共同支撑后续实验分析。

3.3 模型移植与检测模块设计

3.3.1 模型移植流程

本文的目标检测模型首先在 PC 或 WSL 环境中完成训练和导出，然后转换为 RK3588 平台可加载的 RKNN 模型[3,7-9]。整体流程如图 3.2 所示。

图 3.2 模型移植流程图

在模型移植过程中，关键问题不是简单完成格式转换，而是保证模型输出与板端后处理逻辑一致。本文当前使用的稳定模型为单类别 drone 检测模型，板端输出张量形式为 [1, 5, 8400]，其中 5 个通道对应目标框参数和置信度信息。后处理模块需要将模型输出解析为边界框中心点、宽高和置信度，再映射回原始图像坐标系。

对于无人机检测任务，类别数量较少，后处理重点主要集中在置信度阈值、坐标还原和非极大值抑制上。由于目标在远距离情况下通常较小，阈值设置过高容易造成漏检，阈值设置过低则会增加误检。因此，系统将置信度阈值和 NMS 阈值设计为可配置参数，在不同视频场景中进行调试和记录。

3.3.2 检测结果解析

检测模块的输入为预处理后的图像张量，输出为若干候选框。每个候选框包含左上角坐标、右下角坐标、类别编号和置信度。由于当前模型为单类别无人机检测模型，因此类别编号统一映射为 drone。在结果可视化时，系统将类别名称和置信度绘制在检测框附近，便于观察模型输出。

检测结果解析过程包括以下步骤：

1. 读取 RKNN 输出张量，并按照模型输出格式转换为候选框；
2. 根据置信度阈值过滤低置信度候选框；
3. 将网络输入尺寸下的坐标还原到原始图像尺寸；
4. 对重叠候选框执行非极大值抑制；
5. 输出最终检测框，用于绘制、日志保存和后续跟踪。

从设计角度看，检测后处理模块需要保持独立，避免与视频输入、RTSP 输出和多线程调度强耦合。这样做的好处是：当后续更换模型结构、输入尺寸或类别数量时，只需要调整模型加载和后处理逻辑，而不需要重写整个视频处理系统。

3.4 实时视频处理流水线设计

3.4.1 多线程流水线结构

为了提高实时性，本文系统采用多线程流水线设计。系统主要包含采集线程、推理线程和输出线程。采集线程负责从视频源读取帧；推理线程负责执行预处理、NPU 推理和后处理；输出线程负责绘制结果并编码输出视频流。多线程结构如图 3.3 所示。

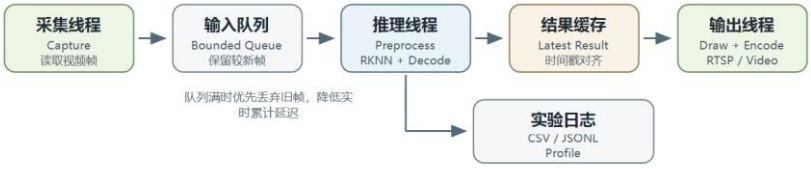

图 3.3 实时视频处理流水线

在该结构中，输入队列采用有限长度设计。当推理速度低于视频输入速度时，系统优先丢弃过旧帧，保留较新的输入帧。这样可以降低实时画面的累计延迟，避免系统为了处理历史帧而导致显示严重滞后。对于实时演示场景，低延迟通常比完整处理每一帧更重要；对于固定视频评测场景，则可以通过离线模式保存完整结果。

3.4.2 实时输出方式

本文系统支持两类输出方式。第一类是固定视频输出，即将检测框绘制到视频文件中，并保存逐帧检测结果。这种方式适合论文实验，因为同一输入视频可以在不同参数配置下重复运行。第二类是实时 RTSP 输出，即将带框画面编码为视频流，电脑端可以通过播放器打开 RTSP 地址进行观看。这种方式适合系统演示和板端联调。

固定视频和实时 RTSP 的定位不同。固定视频用于严谨对比，能够得到稳定的帧率、延迟和检测结果统计；实时 RTSP 用于验证系统是否能在实际部署中持续运行。本文后续实验中，固定视频作为主要对比入口，实时 RTSP 作为最终演示入口。

3.5 动态 ROI 与轻量跟踪策略设计

3.5.1 检测间隔策略

在嵌入式实时检测中，如果每一帧都执行完整 NPU 推理，系统可能出现延迟升

高或视频卡顿。为此，本文设计了检测间隔策略，即每隔 N 帧执行一次完整检测，中间帧复用上一帧检测结果或通过轻量跟踪进行更新。该策略通过参数 detect_every_n 控制。

设当前帧编号为 k，检测间隔为 N，则是否执行完整检测可表示为：

$$\text{Infer}(k)=1,\ k \bmod N = 0;\ \text{Infer}(k)=0,\ k \bmod N \neq 0 \tag{3.1}$$

当 Infer(k)=1 时，系统执行完整 NPU 检测；当 Infer(k)=0 时，系统不调用 NPU，而是根据上一帧结果和跟踪策略生成当前帧的显示框。该方法能够降低 NPU 调用频率，从而改善输出画面的连续性和系统延迟。

3.5.2 动态 ROI 策略

无人机目标在画面中通常较小，并且在相邻帧之间位置变化具有连续性。基于这一特点，系统设计了动态 ROI 策略：当上一帧存在稳定检测结果时，系统可以围绕目标位置生成局部区域，用于后续帧的跟踪、结果复用或局部分析。动态 ROI 的作用不是替代模型检测，而是在检测间隔期间提供目标区域约束，减少检测框跳变。

动态 ROI 的设计需要考虑两个问题。第一，ROI 区域不能过小，否则目标快速运动时容易丢失；第二，ROI 区域不能过大，否则对抑制背景干扰的作用会减弱。因此，系统在检测框周围保留一定扩展边界，并结合图像边界进行裁剪，保证 ROI 始终位于有效图像范围内。

3.5.3 轻量跟踪与框平滑

在实时显示中，即使模型能够正确识别目标，检测框也可能因为输入画面噪声、目标尺寸较小和模型置信度波动而产生抖动。为改善显示效果，本文系统设计了轻量级跟踪和平滑策略。跟踪策略用于在未执行完整检测的帧中估计目标位置；框平滑策略用于减少相邻检测框之间的突变。

框平滑采用指数平滑思想，将当前检测框与上一帧显示框进行加权融合：

$$B_smooth(k)=alpha\ B_detect(k)+(1-alpha)B_smooth(k-1) \tag{3.2}$$

其中，B_detect(k) 表示当前帧检测框，B_smooth(k-1) 表示上一帧平滑后的检测框，alpha 为平滑系数。alpha 越大，检测框越接近当前检测结果；alpha 越小，检测框

越稳定但响应速度越慢。本文在实际测试中将平滑参数设计为可配置项，用于在稳定性和响应速度之间取得平衡。

3.6 多 context NPU 并行推理设计

3.6.1 多 context 设计思想

RK3588 平台的 NPU 具备并行计算能力，但在单线程、单 context 调用方式下，视频预处理、输入设置、NPU 执行和输出读取往往串行发生，无法充分重叠不同帧之间的处理过程。为了提高 NPU 使用效率，本文设计了多 context 并行推理结构，即创建多个 RKNN context，并由多个推理工作线程并行处理不同帧。

多 context 并行推理结构如图 3.4 所示。

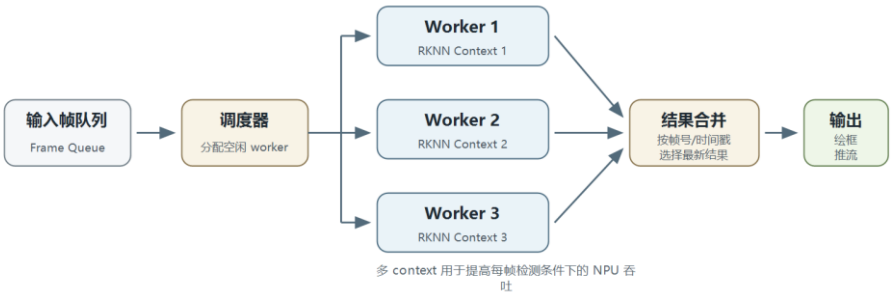

图 3.4 多 context NPU 并行推理结构

在该结构中，每个推理线程拥有独立的 RKNN context，线程之间不共享模型运行状态。输入帧由调度器分配给空闲 worker，推理完成后将结果写入结果队列。输出端根据时间戳或帧编号选择最新结果进行绘制，避免较旧推理结果覆盖较新画面。

3.6.2 多 context 与检测间隔的关系

检测间隔策略和多 context 策略解决的是不同问题。检测间隔通过减少 NPU 调用次数降低系统负载，适合实时演示中对低延迟和稳定显示的需求；多 context 策略通过并行化提高 NPU 推理吞吐，适合分析 RK3588 的硬件能力和每帧检测场景。

在实际设计中，两种策略可以组合使用。例如，当 detect_every_n=2 时，系统每两帧执行一次完整检测，可以获得较好的显示稳定性；当 detect_every_n=1 且启用两个 context 时，系统可以测试每帧检测条件下的最高 NPU 吞吐能力。本文后续实验将

比较单 context、双 context 和多 context 的差异，分析不同配置对 stream_fps、npu_fps 和端到端延迟的影响。

3.6.3 阶段耗时分析设计

为了定位系统瓶颈，本文在推理模块中设计了阶段耗时记录功能。每次推理过程被划分为图像准备、输入设置、NPU 运行、输出读取、检测解码和资源释放等阶段。通过记录各阶段耗时，可以判断瓶颈来自 NPU 计算、输入数据传输、后处理还是视频输出。

阶段耗时分析的意义在于避免盲目优化。例如，如果后处理耗时很低，而输入设置和 NPU 运行耗时占主要比例，则继续优化 NMS 或绘框代码对整体实时性提升有限；此时更值得关注输入格式、数据传输、多 context 调度或模型结构本身。本文后续实验将利用该设计对 RK3588 板端运行过程进行量化分析。

3.7 本章小结

本章完成了基于 RK3588 嵌入式平台的目标检测系统总体设计。首先，从功能需求、性能需求和部署约束三个方面分析了系统需要解决的问题；随后，提出了由视频输入、预处理、NPU 推理、后处理、策略控制和结果输出组成的分层架构；接着，设计了模型移植流程、实时视频处理流水线、动态 ROI 与轻量跟踪策略；最后，给出了面向 NPU 性能优化的多 context 并行推理设计和阶段耗时分析方法。

通过上述设计，系统能够在保证原有检测功能可运行的基础上，进一步支持固定视频评测、实时 RTSP 演示和多线程 NPU 性能对比。下一章将围绕各模块的具体实现展开，包括 RKNN 模型加载、视频流处理、检测后处理、RTSP 输出、多线程调度以及实验日志记录等内容。

第四章 系统实现与调试

第三章完成了基于 RK3588 嵌入式平台的目标检测系统总体设计，本章在此基础上进一步说明系统的具体实现过程。系统实现围绕两个核心目标展开：一是将训练得到的无人机检测模型转换为 RK3588 NPU 可加载的 RKNN 模型，并完成板端推理；二是在板端构建固定视频检测和实时 RTSP 推流两条运行路径，使系统既能够进行可重复实验，又能够满足实时演示需求。

本章所述实现主要包括开发环境配置、模型导出与转换、RKNN 推理封装、固定视频检测程序、实时 RTSP 检测程序、轻量跟踪与动态 ROI 策略、多 context 并行推理以及运行日志记录等内容。与纯算法训练不同，嵌入式部署需要同时处理模型格式、输入输出张量、视频解码、线程同步、推流显示和性能统计等工程问题。因此，本章重点从工程实现角度说明系统各模块如何协同工作，并为第五章的实验测试和性能分析提供实现基础。

4.1 开发与部署环境

系统开发过程分为 PC/WSL 端和 RK3588 板端两部分。PC/WSL 端主要用于模型训练、模型导出、RKNN 转换准备和代码整理；RK3588 板端主要用于实际模型加载、视频输入、NPU 推理、检测结果绘制和 RTSP 输出。这样的分工可以降低板端开发压力，使耗时较长的模型训练和格式转换尽量在 PC 端完成，而把板端工作集中在实时推理和系统集成上。

表 4.1 给出了本文系统实现所涉及的主要软硬件环境。

表 4.1 系统开发与部署环境

类别	环境或工具	作用说明
开发主机	Windows 与 WSL 环境	用于代码管理、模型训练脚本运行、ONNX/RKNN 转换准备和论文资料整理
嵌入式平台	RK3588 开发板	作为最终部署平台，利用板载 NPU 执行目标检测推理
目标检测模型	YOLOv10 单类无人机检测模型	作为系统目标检测算法主体，输出无人机候选框和置信度
推理框架	RKNN Runtime	在 RK3588 板端加载 RKNN 模型并调用 NPU 推理
视频处理库	OpenCV	用于视频文件读取、USB 摄像头读取、图像缩放、颜色转换和检测框绘制
推流组件	GStreamer RTSP Server	用于将检测后的实时画面发布为 RTSP 视频流
构建工具	CMake 与 Make	用于在 RK3588 板端编译 C/C++ 程序
实验记录	CSV、JSONL 和运行日志	用于保存检测结果、ROI 信息、FPS、延迟和阶段耗时

在板端运行时，系统采用 C/C++ 作为主要实现语言。其原因在于 C/C++ 便于直接调用 RKNN Runtime、OpenCV 和 GStreamer 等底层库，能够更好地控制线程、队列和内存使用。同时，C/C++ 程序在嵌入式平台上运行开销较低，更适合实时视频处理场景。

4.2 模型导出与 RKNN 转换实现

本文使用的无人机检测模型首先在 PC/WSL 环境中完成训练，得到 PyTorch 权重文件。为了在 RK3588 上运行，需要将模型从 PyTorch 格式转换为 ONNX 格式，再由 RKNN Toolkit2 转换为 RKNN 模型。整体流程为：

```
best.pt -> best.onnx -> best.rk3588.fp.rknn -> RK3588 NPU 推理
```

在模型转换过程中，本文优先采用 FP 类型 RKNN 模型作为稳定部署版本。虽然 INT8 量化理论上可以进一步降低计算和存储开销，但量化需要校准数据集、量化误差评估和板端精度验证。当前系统已经稳定跑通 FP 模型，因此本章将 FP RKNN 作为主实现路径，INT8 量化仅作为后续优化方向保留，不将其写成已经完整验证的结果。

表 4.2 给出了模型迁移过程中涉及的主要文件类型及其作用。

表 4.2 模型文件及作用

文件类型	示例文件	作用说明
PyTorch 权重	best.pt	训练完成后得到的原始模型权重，用于继续训练、验证和导出
ONNX 模型	best.onnx	用于跨框架转换的中间模型格式，便于导入 RKNN Toolkit2
RKNN 模型	best.rk3588.fp.rknn	RK3588 板端实际加载的模型文件，由 RKNN Runtime 调用
检测输出	[1, 5, 8400]	单类别无人机检测模型的输出张量，包含边界框参数和置信度

为了适配单类无人机检测任务，后处理模块不再使用 COCO 多类别标签表作为唯一判断依据，而是根据模型输出通道数识别当前模型类别数。当输出张量为 [1, 5, 8400] 时，系统将其解析为单类别检测输出，其中 5 个通道分别对应候选框中心点、宽高和置信度信息。检测结果在可视化时统一标注为 drone，避免出现类别映射错误。

4.3 RKNN 推理模块实现

RKNN 推理模块是系统的核心计算模块，负责完成模型加载、输入预处理、NPU 推理、输出读取和检测结果解析[7]。本文将该部分封装为独立的检测器类，使固定视频检测程序和实时 RTSP 程序可以复用相同的推理逻辑。

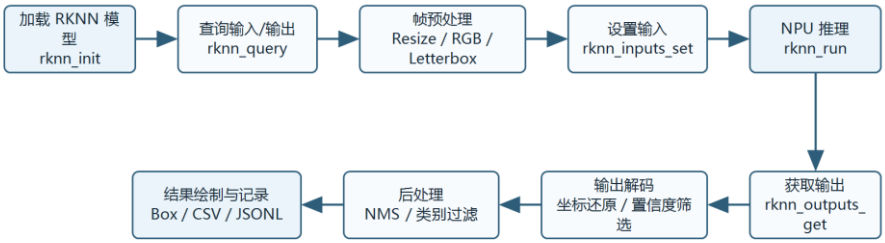

图 4.1 RKNN 推理模块实现流程

RKNN 推理模块的主要步骤如下：

1. 调用 rknn_init 加载 RKNN 模型文件；
2. 通过 rknn_query 查询模型输入输出张量属性；
3. 根据输入张量格式确定模型输入宽度、高度、通道数和数据排布；
4. 对原始图像进行缩放、补边和颜色空间转换；
5. 调用 rknn_inputs_set 设置模型输入；
6. 调用 rknn_run 在 NPU 上执行模型推理；
7. 调用 rknn_outputs_get 获取输出张量；
8. 对输出张量执行候选框解析、置信度过滤和非极大值抑制；
9. 将检测框坐标映射回原始图像尺寸；
10. 释放输出内存并返回检测结果。

在实现中，系统会在模型加载阶段打印输入输出张量属性，包括张量维度、数据类型和存储格式。这样做的目的是在模型转换后第一时间发现输入尺寸不匹配、输出通道数异常或数据布局不一致等问题。对于 YOLO 系列模型而言，输出张量格式可能随导出参数而变化，如果不在板端进行显式检查，容易导致后处理阶段解析错误。

表 4.3 给出了 RKNN 推理模块中几个关键阶段的功能划分。

表 4.3 RKNN 推理模块关键阶段

阶段	对应功能	实现重点
模型初始化	加载 RKNN 文件并查询张量属性	判断输入尺寸、通道数和输出布局是否符合预期
图像预处理	缩放、补边、颜色转换	保持与训练和导出阶段一致的输入格式
输入设置	将预处理后的图像写入 RKNN 输入	当前稳定路径使用常规输入设置方式
NPU 推理	执行 rknn_run	由 RK3588 NPU 完成主要神经网络计算
输出解析	候选框解码、阈值筛选和 NMS	支持单类无人机检测输出格式
坐标还原	将模型坐标映射回原图	保证绘制框位置与原始画面一致

为了便于后续性能分析，推理模块还提供了可选的阶段耗时统计接口。开启 profiling 后，系统可以分别记录图像准备、输入设置、NPU 执行、输出读取、解码和释放等阶段耗时。该接口在正常演示时默认关闭，以减少日志输出和额外开销；在实验分析时再通过环境变量开启。

4.4 固定视频检测程序实现

固定视频检测程序 rk_yolo_video 是本文最先跑通的 RK3588 验证路径。该程序的作用是在板端读取本地视频文件，逐帧执行 RKNN 推理，将检测框绘制到原始画面中，并输出带框视频、检测结果 CSV 和 ROI JSONL 文件。

该程序的命令行接口如下：

```
./rk_yolo_video <input_video> <output_video> [model_path] [score_thresh] [nms_thresh] [detections_csv] [roi_jsonl]
```

其中，input_video 为输入视频文件，output_video 为带框输出视频，model_path 为 RKNN 模型路径，score_thresh 和 nms_thresh 分别为置信度阈值和非极大值抑制阈值。如果没有显式指定 CSV 和 JSONL 文件路径，程序会根据输出视频文件名自动生

成对应的检测记录文件。

固定视频检测程序的输出包括三类数据：

1. 带框视频，用于直观看到模型是否检测到目标；
2. CSV 检测记录，用于统计每一帧的检测结果；
3. ROI JSONL 文件，用于兼容后续动态 ROI 或旧系统输出格式。

CSV 文件每一行对应一个检测框，字段包括帧序号、类别编号、类别名称、置信度、左上角坐标、宽度和高度。其字段格式如下：

```
frame_index,class_id,class_name,score,left,top,width,height
```

ROI JSONL 文件则按帧保存检测结果，每一行是一个 JSON 对象，便于后续程序按帧读取目标区域信息。固定视频路径具有较强的可重复性，同一段视频、同一模型和同一阈值可以反复运行得到可比较结果，因此本文将其作为板端固定输入实验的主要入口。

4.5 实时 RTSP 检测程序实现

在固定视频检测路径稳定后，本文进一步实现了实时 RTSP 检测程序 rk_yolo_live_rtsp。该程序既支持 USB 摄像头输入，也支持本地视频文件循环输入。系统读取视频帧后执行目标检测，将检测框叠加到画面上，再通过 GStreamer RTSP Server 发布为 RTSP 视频流，使 PC 端可以使用播放器或 GStreamer 客户端实时查看检测画面。

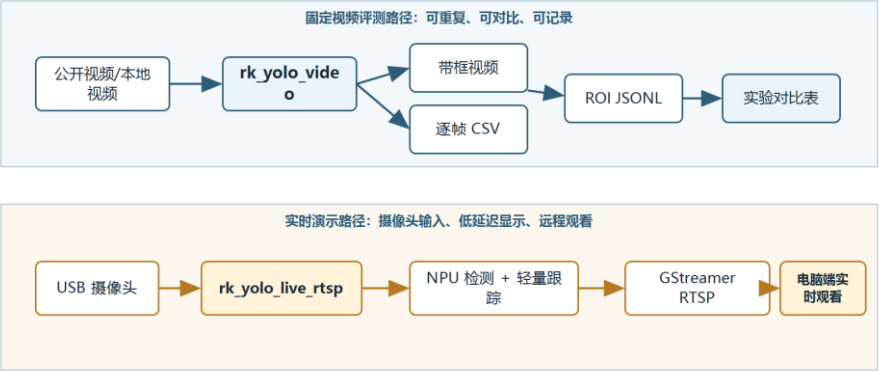

图 4.2 固定视频与实时 RTSP 两条运行路径

实时 RTSP 程序的基本命令行接口如下：

```
./rk_yolo_live_rtsp [device] [model] [mount] [width] [height] [fps] [score] [nms] [port] [detect_every_n]
```

其中，device 可以是 /dev/video* 摄像头设备，也可以是本地视频文件路径；model 为 RKNN 模型路径；mount 为 RTSP 挂载路径；width、height 和 fps 用于指定输出视频参数；score 和 nms 用于设置检测阈值；detect_every_n 用于控制每隔多少帧执行一次完整 NPU 检测。

实时 RTSP 程序内部采用三阶段流水线结构：

1. 采集线程负责读取摄像头或视频文件，并将最新帧写入有界队列；
2. 推理线程负责从队列中取出图像，根据调度策略执行 NPU 检测或复用跟踪结果；
3. 发布线程负责绘制检测框，将画面推送给 RTSP 服务端。

采用有界队列的原因是实时视频系统不能无限积压历史帧。如果推理速度低于输入帧率，系统应优先保证输出画面接近当前时刻，而不是依次处理所有过期帧。因此，队列长度被设计得较小，在必要时丢弃旧帧，以降低端到端延迟。

RTSP 输出基于 GStreamer 实现。程序内部创建 appsrc，将绘制后的图像帧推入 GStreamer 管线，再通过 H.264 编码器和 RTSP Server 对外发布。PC 端可以通过如下地址查看检测画面：

```
rtsp://<board-ip>:8554/drone
```

对于固定视频循环演示，可以设置 RK_YOLO_INPUT_LOOP=1，使程序在视频播放结束后自动回到开头，便于长时间观察和演示。在 USB 摄像头输入场景下，程序还支持对特定 UVC 摄像头执行启动参数调整，例如设置焦距、关闭自动对焦和设置变焦值，以提高画面稳定性。

4.6 检测间隔、动态 ROI 与轻量跟踪实现

在实时检测场景中，如果每一帧都执行完整 NPU 推理，系统可能出现延迟升高或队列积压。为此，本文实现了检测间隔策略，即通过 detect_every_n 参数控制完整检测频率。当 detect_every_n=1 时，系统每帧执行一次 NPU 检测；当该参数大于 1 时，系统仅在关键帧执行完整检测，其余帧根据轻量跟踪结果更新显示框。

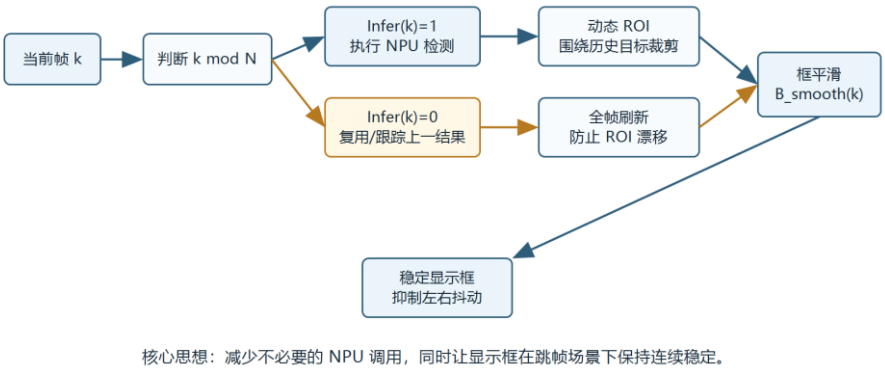

图 4.3 跳帧检测、动态 ROI 与检测框平滑策略

检测间隔策略可表示为：

$$\text{Infer}(k)=1,\ k \bmod N = 0;\ \text{Infer}(k)=0,\ k \bmod N \neq 0 \tag{4.1}$$

其中，k 表示当前帧序号，N 表示检测间隔。当 Infer(k)=1 时，系统执行完整 NPU 检测；当 Infer(k)=0 时，系统不调用 NPU，而是使用跟踪或上一帧检测结果进行显示更新。

为了减少跳帧检测导致的检测框抖动，系统实现了轻量框平滑策略。其基本思想

是将当前检测框与上一帧显示框进行加权融合：

$$B_smooth(k)=alpha\ B_detect(k)+(1-alpha)B_smooth(k-1) \tag{4.2}$$

其中，B_detect(k) 表示当前检测框，B_smooth(k-1) 表示上一帧平滑后的显示框，alpha 为平滑系数。alpha 越大，显示框越接近当前检测结果；alpha 越小，显示框越稳定但响应速度越慢。系统通过环境变量提供该参数，使不同场景下可以根据目标运动速度和画面抖动程度进行调节。

除框平滑外，系统还实现了动态 ROI 策略。当上一帧已经检测到目标时，下一次关键帧推理可以优先围绕历史目标区域构造裁剪窗口，并加入一定边界扩展。这样可以在目标较小的情况下提高局部区域内的目标占比，同时减少无关背景对检测的干扰。为了避免 ROI 长时间漂移，系统设置了周期性全帧刷新机制，使检测过程能够重新回到完整画面中恢复目标。

表 4.4 给出了实时检测策略相关的主要配置项。

表 4.4 实时检测策略配置项

配置项	默认或推荐含义	作用说明
detect_every_n	控制完整检测间隔	在 NPU 调用频率和显示稳定性之间折中
RK_YOLO_TRACK_MODE	轻量运动跟踪模式	在非关键帧估计检测框位置
RK_YOLO_BOX_SMOOTH	是否启用框平滑	减少检测框在相邻帧之间的突变
RK_YOLO_BOX_SMOOTH_ALPHA	平滑系数	控制检测框响应速度与稳定性的平衡
RK_YOLO_DYNAMIC_ROI	是否启用动态 ROI	根据历史检测区域裁剪下一次推理输入
RK_YOLO_ROI_REFRESH	ROI 刷新间隔	定期恢复全帧检测，降低跟踪漂移风险

4.7 多 context NPU 并行推理实现

为了满足硬件优化方向的研究需求，本文在实时 RTSP 程序中实现了实验性的多 context NPU 并行推理机制。该机制的核心思想是在程序中创建多个 RKNN 检测器实例，每个实例拥有独立的 RKNN context，并由多个推理工作线程并行处理不同帧。

多 context 模式仅在 detect_every_n=1 时启用。这一限制是有意设计的：当系统已经通过 detect_every_n>1 降低 NPU 调用频率时，多 context 并行推理的收益并不明显，反而可能增加调度复杂度。只有在每帧都需要完整检测时，多 context 才能更直接地体现 NPU 并行吞吐能力。

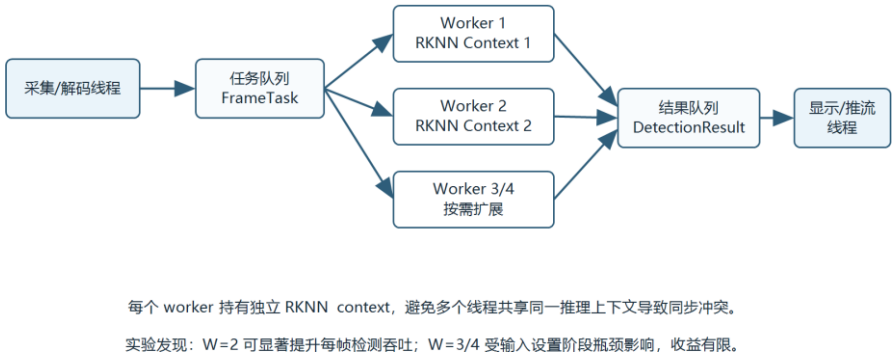

图 4.4 多 context NPU 并行推理实现

多 context 推理的实现流程如下：

1. 主线程读取 RK_YOLO_INFER_WORKERS 环境变量，确定推理工作线程数量；
2. 为每个工作线程创建独立的 RKNN 检测器和 RKNN context；
3. 采集线程将最新帧送入调度队列；
4. 调度逻辑将待检测帧分配给空闲 worker；
5. worker 完成 NPU 推理后将结果写入结果队列；
6. 发布线程根据帧序号和时间戳选择最新可用结果进行绘制。

表 4.5 给出了单 context 和多 context 实现方式的对比。

表 4.5 单 context 与多 context 实现对比

实现方式	特点	适用场景
单 context	资源占用较低，调度简单，延迟更容易控制	实时演示和稳定运行
双 context	可以重叠多个推理任务，提高每帧检测吞吐	NPU 并行能力验证
三 context 及以上	调度和内存开销进一步增加，收益需要实验确认	扩展性分析，不作为默认配置

多 context 机制使本文不仅能够完成“模型在 RK3588 上跑通”的基本目标，还能够围绕导师提出的 NPU 多线程方向开展实验。需要注意的是，多 context 的目标并不

是简单替代当前实时演示配置，而是提供一条硬件并行优化路线。后续实验需要同时比较吞吐、延迟、CPU 占用和内存占用，才能判断其实际价值。

4.8 日志记录与性能调试接口

为了支撑论文实验分析，系统在实现过程中加入了多层日志记录机制。固定视频检测程序主要输出检测结果 CSV、ROI JSONL 和带框视频；实时 RTSP 程序则输出运行时 FPS、NPU 推理频率、端到端延迟、工作线程耗时和队列状态等信息。

实时程序的常规日志包括：

1. stream_fps：实际发布的视频帧率；
2. npu_fps：实际执行 NPU 推理的频率；
3. roi_fps：动态 ROI 推理频率；
4. work_ms：单帧主要处理耗时；
5. 队列丢帧信息：用于判断系统是否出现输入积压；
6. 当前配置项：包括检测间隔、平滑参数、worker 数量和 ROI 开关。

当需要进行更细粒度性能分析时，可以设置：

```
RK_YOLO_PROFILE=1
```

开启该开关后，程序会输出推理阶段的细分耗时，包括预处理、输入设置、NPU 运行、输出读取、检测解码和绘制等。通过这些指标，可以判断系统瓶颈究竟来自 NPU 计算、输入数据传输、后处理还是视频输出。

系统还保留了实验性的 zero-copy input 路径，对应环境变量为：

```
RK_YOLO_ZERO_COPY_INPUT=1
```

该路径用于探索 RKNN 输入内存绑定是否能够降低输入设置开销。但根据当前工程状态，zero-copy 仍属于实验功能，默认关闭。本文在系统实现章节中仅说明其作为调试接口存在，不将其作为稳定主路径。这样的写法可以真实反映项目进展，避免把尚未完整验证的优化方向写成成熟结果。

4.9 本章小结

本章围绕 RK3588 嵌入式平台上的目标检测系统实现过程进行了说明。首先介绍了 PC/WSL 端和 RK3588 板端的开发部署环境；随后说明了从 PyTorch 权重到 ONNX，再到 RKNN 模型的迁移过程，并明确当前稳定主路径为 FP RKNN 模型。接着，本章详细介绍了 RKNN 推理模块的封装方式，包括模型加载、输入预处理、NPU 推理、输出解析和坐标还原等步骤。

在系统运行层面，本章分别介绍了固定视频检测程序 rk_yolo_video 和实时 RTSP 检测程序 rk_yolo_live_rtsp 的实现。固定视频检测程序用于可重复实验和模型验证，实时 RTSP 程序用于实时观看和演示。针对实时场景中的延迟和检测框抖动问题，本章说明了检测间隔、轻量跟踪、框平滑和动态 ROI 等策略的实现方式。同时，为满足 NPU 多线程优化方向，本章实现了多 context 并行推理机制，并通过日志和 profiling 接口为后续实验分析提供数据基础。

通过本章实现，系统已经具备在 RK3588 板端运行无人机目标检测模型、输出带框视频、实时 RTSP 推流以及记录性能数据的能力。下一章将在此基础上，对固定视频检测、实时 RTSP 演示、检测间隔、框平滑和多 context 推理等配置进行实验测试与结果分析。

第五章 系统测试与实验分析

前两章分别完成了系统总体设计和板端实现说明。本章在此基础上对系统进行实验测试与结果分析，重点验证三类问题：第一，训练得到的单类无人机检测模型是否能够在公开无人机视频中产生有效检测结果；第二，模型转换为 RKNN 后能否在 RK3588 开发板上稳定运行；第三，不同实时检测策略和多 context NPU 并行方案对系统吞吐、延迟和显示稳定性的影响。

由于无人机真实飞行测试场景受场地、天气和安全条件限制，本文采用“公开无人机视频固定输入 + RK3588 板端实时 RTSP 验证”的实验路线。固定视频实验用于保证输入可重复，便于比较不同配置；实时 RTSP 实验用于验证系统在实际部署路径中的端到端表现。这样可以在缺少长期真实飞行场景的情况下，仍然形成较完整的模型有效性、板端可运行性和系统优化效果证据。

5.1 实验环境与评价指标

本文实验主要包括 PC/WSL 端模型训练与评估、RK3588 板端固定视频验证以及 RK3588 实时 RTSP 推流验证。PC/WSL 端用于训练 YOLOv10 单类无人机模型，并完成 PyTorch、ONNX 与 RKNN 文件的生成；RK3588 板端用于加载 RKNN 模型并执行视频检测、结果绘制和 RTSP 输出。

表 5.1 给出了本章实验涉及的主要环境和输入条件。

表 5.1 实验环境与输入条件

类别	内容	说明
训练主机	Windows 与 WSL 环境	用于 YOLOv10 模型训练、ONNX 导出和 RKNN 转换准备
部署平台	RK3588 开发板	用于加载 RKNN 模型并调用 NPU 推理
部署程序	rk_yolo_video、rk_yolo_live_rtsp	分别用于固定视频检测和实时 RTSP 检测
主要模型	FP RKNN 模型	当前稳定的无人机单类 RKNN 模型
输入视频	公开视频样本	用于短视频验证和较长序列验证
输出内容	带框视频、CSV、ROI JSONL、RTSP 流、运行日志	用于定性观察和定量分析

评价指标分为模型检测指标和系统运行指标两类。模型检测指标主要包括 precision、recall、mAP50 和 mAP50-95，用于反映模型在验证集上的检测效果。系统运行指标主要包括 stream_fps、npu_fps、端到端延迟、CPU 占用、内存占用和丢帧数量，用于评价实时检测系统在 RK3588 上的运行表现。

表 5.2 给出了本章使用的主要系统指标含义。

表 5.2 系统运行指标说明

指标	含义	分析作用
stream_fps	实际输出视频流帧率	反映观看端画面流畅程度
npu_fps	实际执行 NPU 推理的频率	反映模型检测刷新强度
端到端延迟	从采集到输出的整体延迟	反映实时响应能力
work_ms	单帧主要处理耗时	用于分析处理链路负载
CPU 占用	程序运行时 CPU 使用比例	反映板端资源压力
RSS 内存	程序常驻内存占用	反映多 context 和缓存开销
dropped_capture / dropped_publish	采集或发布阶段丢帧数量	反映系统是否出现队列积压

需要说明的是，本文当前阶段的重点是嵌入式部署与 NPU 多线程优化，而不是提出新的检测网络结构。因此，本章在讨论模型效果时更强调“模型能否作为系统输入稳定工作”，在讨论系统实验时则更强调“RK3588 上不同运行策略的吞吐与延迟差异”。

5.2 无人机检测模型训练结果

本文使用 YOLOv10n 作为基础模型，并在单类无人机数据集上进行迁移训练。训练轮数为 50 epoch，输入尺寸为 640，batch size 为 16，训练完成后得到 PyTorch 权重文件 best.pt。该模型随后被导出为 ONNX，并进一步转换为 RK3588 可加载的 RKNN 模型。

表 5.3 给出了训练完成后验证集上的主要指标。

表 5.3 单类无人机模型训练结果

指标	数值
Precision	0.8291
Recall	0.8426
mAP50	0.8901
mAP50-95	0.5095
训练轮数	50
输入尺寸	640 × 640

由表 5.3 可知，当前模型在验证集上具有较高的 mAP50，说明其能够较好地区分无人机目标与背景。mAP50-95 相对低于 mAP50，说明在更严格的定位阈值下，模型的边界框精度仍有提升空间。这与小目标无人机检测任务特点一致：远距离无人机在图像中尺寸较小，轻微的边界框偏移就会对高 IoU 阈值下的评价结果产生较大影响。

在论文表述中，本文不将该训练结果描述为算法创新，而是将其作为后续 RK3588 部署和实时检测实验的模型基础。这样可以避免夸大模型本身贡献，同时突出本课题的系统移植和硬件优化重点。

5.3 公开视频固定输入验证

为了弥补真实无人机飞行场景不易获得的问题，本文引入公开无人机视频作为固定输入测试源。公开视频层的作用不是直接替代完整数据集训练，而是为系统提供可重复的视频输入，使不同程序、不同模型和不同运行策略可以在相同场景下进行比较。

PC/WSL 端首先使用 best.pt 对 anti_uav_fig1.mp4 进行离线检测。该视频共 130 帧，在置信度阈值 0.35 下，检测结果为：16 帧存在检测框，总检测数为 18，最高置信度为 0.6216。这说明当前模型能够在新的公开无人机视频源中产生目标响应。

随后，本文将模型部署到 RK3588 板端进行固定视频验证。早期直接转换的无人机 RKNN 模型曾在 rknn_run 阶段出现运行崩溃。后续通过重新导出 end2end=False 的 ONNX 模型，并在 C++ 后处理模块中适配单类输出头 $[1, 5, 8400]$，恢复了板端运行路径。该过程说明模型部署并不是单纯格式转换，还需要与输出张量结构和后处理逻

辑保持一致。

表 5.4 给出了 RK3588 固定视频验证结果。

表 5.4 RK3588 固定视频检测结果

视频样本	帧数	有检测帧	检测数	平均推理
短公开视频	130	30	30	109.43 ms
DUT video01	1050	846	846	67.61 ms

由表 5.4 可知，恢复后的无人机 RKNN 模型不仅能够在短视频上运行，也能够在较长的 DUT-Anti-UAV 序列上稳定产生检测结果。其中 video01.mp4 共 1050 帧，有 846 帧输出检测框，说明该视频适合作为后续重复实验和论文定性展示的主要公开视频样本。

固定视频验证的意义在于，它将模型可用性、RKNN 运行稳定性和输出记录能力统一到同一条可重复路径中。相比只进行实时摄像头演示，固定视频可以保存带框视频、逐帧 CSV 和 ROI JSONL 文件，更适合论文实验记录和后续误差分析。

5.4 实时 RTSP 首轮配置对比

在固定视频路径跑通后，本文进一步测试实时 RTSP 检测程序。首轮实时实验比较了基线全帧检测、多 context 全帧检测和策略优化检测三类配置。其目标不是单纯追求最高帧率，而是观察不同配置在吞吐和延迟之间的差异。

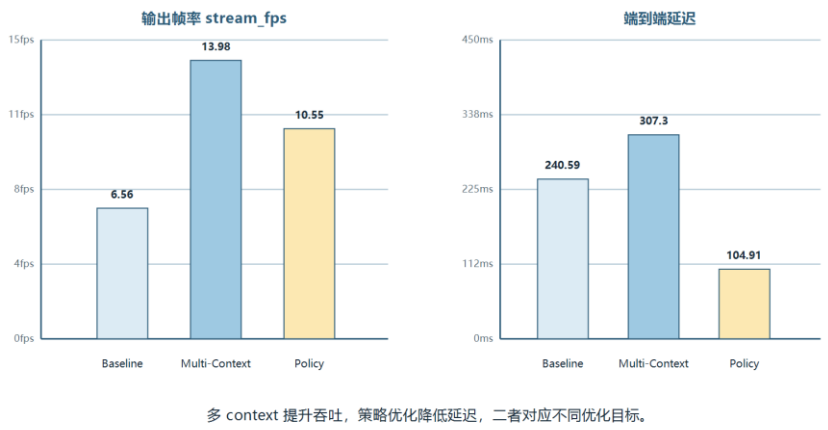

图 5.1 实时 RTSP 首轮配置对比

表 5.5 给出了首轮实时 RTSP 实验的主要结果。

表 5.5 实时 RTSP 首轮配置实验结果

配置	stream	npu	工作耗时	端到端延迟	采集丢帧	说明
Baseline	6.56	6.56	152.49 ms	240.59 ms	80	单 worker 全帧推理基线
Multi-Context	13.98	13.98	180.16 ms	307.30 ms	0	双 context 全帧推理，吞吐最高
Policy-Optimized	10.55	5.04	0.00 ms	104.91 ms	1	跳帧与复用策略，延迟最低

表中 Policy-Optimized 的平均推理/工作耗时显示为 0.00 ms，是因为采样日志对应的是复用帧，而不是完整 NPU 推理帧。因此该数值不能直接与全帧推理配置比较，但其端到端延迟仍然反映了策略模式下的实时观看响应。

实验结果显示，Multi-Context 配置提高了输出帧率和 NPU 推理频率，但端到端延迟也随之升高。Policy-Optimized 配置没有追求每帧都执行完整检测，而是通过跳帧检测、动态 ROI 和轻量跟踪保持显示连续性，端到端延迟最低。该结果说明实时系统优化不能只看 NPU 吞吐，还需要综合考虑检测刷新频率、队列积压、输出延迟和观看体验。

5.5 检测间隔与框平滑策略实验

在实时观看过程中，检测框曾出现左右抖动现象。分析认为，这与检测间隔较大、轻量跟踪漂移以及框平滑未启用有关。为此，本文比较了 detect_every_n 和 box smoothing 对实时显示稳定性的影响。

首先，在无目标或弱目标实时场景下比较了 detect_every_n=2、3、4 三种策略。实验结果如表 5.6 所示。

表 5.6 检测间隔策略实验结果

配置	stream	npu	延迟	CPU	RSS	说明
N=2	10.02	6.68	175.20 ms	43.0%	132.37 MB	NPU 刷新强度较高，但延迟较高
N=3	10.91	4.96	107.18 ms	37.1%	127.36 MB	帧率、延迟和资源占用较均衡
N=4	9.54	3.82	95.54 ms	33.5%	127.97 MB	延迟较低，但刷新强度下降

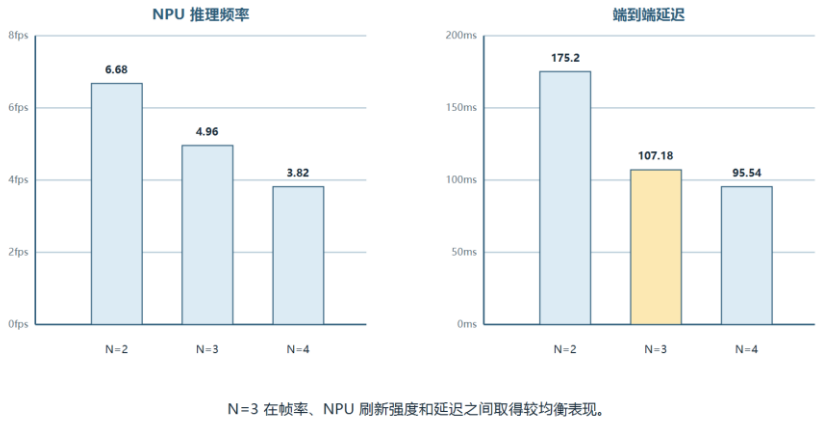

图 5.2 检测间隔策略性能变化

由表 5.6 可知，随着 detect_every_n 增大，NPU 推理频率降低，端到端延迟总体下降，但目标刷新能力也随之降低。N=4 虽然延迟较低，但输出帧率和检测刷新强度均下降；N=2 虽然检测更频繁，但延迟较高。因此，在该轮实验中 N=3 可作为无目标实时场景下的较优折中点。

后续在公开视频 video01.mp4 的目标场景中再次比较 N=3 和 N=2，并加入框平滑。结果表明，N=2 配合 box_smooth=on 后可以减轻检测框左右抖动，虽然 NPU 调用频率提高，但输出帧率仍保持在约 10 FPS。表 5.7 给出了该轮消融实验结果。

表 5.7 跳帧检测与框平滑消融实验结果

配置	stream	npu	延迟	工作耗时	观察结果
N=3，平滑关闭	9.95	3.32	102.60 ms	101.99 ms	框抖动较明显
N=3，平滑开启	9.93	3.31	108.05 ms	107.03 ms	抖动减轻，但刷新频率不变
N=2，平滑开启	10.36	5.18	111.41 ms	110.76 ms	检测框更稳定，实时性仍可接受

综合两轮实验可知，若系统处于纯实时观看、目标不明显或演示场景较简单时，detect_every_n=3 可以作为低延迟默认配置；若视频中存在无人机目标且需要更稳定的框位置，detect_every_n=2 加框平滑更适合作为当前演示配置。

5.6 多 context NPU 并行推理实验

导师提出的主要优化方向是利用 RK3588 NPU 做多线程并行以提高效率。为验证这一方向，本文在实时 RTSP 程序中加入多 context 推理机制，即创建多个 RKNN 检测器实例，每个 worker 拥有独立 RKNN context，并由调度队列分配待检测帧。

表 5.8 给出了多 context NPU 并行实验结果。

表 5.8 多 context NPU 并行推理实验结果

配置	N	W	stream	npu	延迟	CPU	RSS	丢帧
A	2	1	10.37	5.18	109.06 ms	121.5%	205.5 MB	0 capture / 0 publish
B	1	1	8.59	8.59	268.89 ms	144.7%	206.3 MB	116 capture / 0 publish
C	1	2	10.63	10.64	173.58 ms	143.4%	233.7 MB	0 capture / 0 publish
D	1	3	10.45	10.45	179.59 ms	147.7%	239.0 MB	0 capture / 0 publish

实验结果表明，单 context 每帧检测并不适合作为实时默认配置。虽然其 NPU 推理频率高于跳帧策略，但 stream_fps 下降到 8.59 FPS，端到端延迟升高到 268.89 ms，并出现采集丢帧。双 context 每帧检测将 NPU FPS 提升到 10.64 FPS，同时消除了采集丢帧，表明多 context 机制提高了每帧检测场景下的吞吐能力。

然而，三 context 并没有进一步提升性能，反而在延迟和内存占用方面略差于双 context。这说明在当前系统中，两个 context 之后瓶颈已经不再只是 NPU context 数量，输入设置、内存传输、线程调度和输出发布等环节开始限制整体性能。

因此，多 context 实验可形成两条结论。第一，从硬件优化角度看，双 context 是本实验配置下较有效的 NPU 并行配置，实验结果表明 RK3588 NPU 多 context 并行对吞吐有实际提升。第二，从实时演示角度看，detect_every_n=2 或 detect_every_n=3 的策略模式仍然更适合作为低延迟观看配置。

5.7 阶段耗时与瓶颈分析

为了进一步解释为什么三 context 收益有限，本文加入 RK_YOLO_PROFILE 开关，对推理阶段进行细粒度计时。该开关可以记录 prepare、rknn_inputs_set、rknn_run、outputs_get、decode 和 render 等阶段耗时。

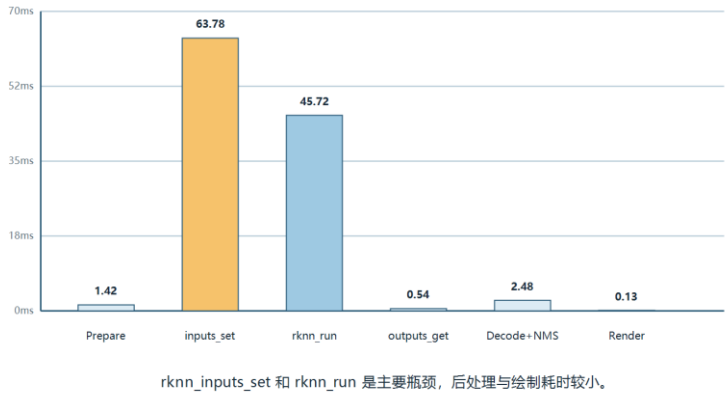

图 5.3 RKNN 推理阶段耗时分布

表 5.9 给出了单 context 和双 context 条件下的阶段耗时对比。

表 5.9 RKNN 推理阶段耗时分析

阶段	N=1，W=1	N=1，W=2
Stream FPS	8.57	10.53
NPU FPS	8.57	10.53
端到端延迟	255.52 ms	179.52 ms
总推理工作耗时	114.48 ms	113.96 ms
Prepare	3.62 ms	1.42 ms
rknn_inputs_set	62.01 ms	63.78 ms
rknn_run	45.82 ms	45.72 ms
rknn_outputs_get	0.49 ms	0.54 ms
Decode + NMS	2.40 ms	2.48 ms
Render	0.15 ms	0.13 ms

由表 5.9 可知，双 context 并没有降低单次推理本身的总工作耗时，两个配置下总推理耗时都约为 114 ms。双 context 的提升来自不同推理任务之间的重叠执行，而不是单帧推理速度变快。

更重要的是，rknn_inputs_set 的耗时约为 62 至 64 ms，甚至高于 rknn_run 的约 46

ms。这表明当前瓶颈并不在后处理或绘制阶段。Decode + NMS 仅约 2.4 ms，Render 仅约 0.1 至 0.2 ms，继续优化画框或标签绘制对整体吞吐帮助有限。后续若要进一步提升性能，应优先考虑输入数据传输、预处理和内存布局优化。

5.8 Zero-copy 输入路径探索

基于阶段耗时分析，本文进一步尝试 RKNN zero-copy 输入路径，希望降低 rknn_inputs_set 的输入设置开销。实验通过 RK_YOLO_ZERO_COPY_INPUT 环境变量控制，默认关闭，以避免影响稳定演示路径。

在 zero-copy 实验中，程序使用 rknn_create_mem 创建 RKNN 管理的输入内存，并通过 rknn_set_io_mem 将其绑定到模型输入。实验结果如表 5.10 所示。

表 5.10 Zero-copy 输入实验结果

指标	标准路径	Zero-copy
Stream FPS	8.57	8.57
NPU FPS	8.57	8.57
端到端延迟	255.52 ms	267.19 ms
总推理工作耗时	114.48 ms	118.93 ms
输入设置或更新	62.01 ms	0.43 ms
rknn_run	45.82 ms	111.36 ms
Decode + NMS	2.40 ms	2.64 ms

由表 5.10 可知，zero-copy 路径将显式输入更新耗时从约 62 ms 降低到 1 ms 以下，但 rknn_run 的耗时从约 46 ms 上升到约 111 ms，最终总耗时并未下降。这说明输入转换或同步开销并没有真正消失，而是从 rknn_inputs_set 阶段转移到了 rknn_run 阶段。

造成这一现象的可能原因是当前模型输入张量与应用侧准备的数据类型并不完全匹配。例如应用侧仍以 uint8 图像作为输入，而模型或 RKNN Runtime 内部仍需要执行格式或类型转换。因此，简单绑定 RKNN 输入内存并不足以得到真正的零拷贝加速。

该实验虽然没有带来性能提升，但对论文仍然有价值。它说明本文的优化过程不是只停留在经验调参，而是通过阶段计时定位瓶颈，并对输入内存路径进行验证。当前稳定演示路径应继续保持 RK_YOLO_ZERO_COPY_INPUT=0，zero-copy 与 RGA 硬件预处理可作为后续优化方向。

5.9 综合讨论与推荐配置

综合本章实验，可以得到以下结论。

第一，当前单类无人机模型已经能够作为 RK3588 检测系统的有效输入。训练结果显示 mAP50 达到 0.8901，公开无人机视频验证也表明模型可以在新场景中产生检测框。板端固定视频实验进一步表明，恢复后的 end2end=False RKNN 模型可以在

RK3588 上稳定运行。

第二，固定视频和实时 RTSP 两条路径具有不同实验价值。固定视频路径可重复性强，适合模型可用性验证、输出视频生成和 CSV 结果分析；实时 RTSP 路径更接近实际部署，适合评价端到端延迟、观看流畅性和系统稳定性。

第三，RK3588 NPU 多 context 并行对每帧检测吞吐具有提升作用。相比单 context 每帧检测，双 context 将 NPU FPS 从 8.59 提升到 10.64，并消除了采集丢帧。但三 context 没有继续带来收益，说明系统已出现输入传输和调度瓶颈。

第四，实际演示配置不应只追求最高 NPU FPS。多 context 全帧检测吞吐更高，但端到端延迟较大；策略优化模式通过跳帧检测、动态 ROI 和框平滑获得更好的观看响应。因此，系统级推荐配置和硬件优化实验配置应分开表述。

表 5.11 给出了当前阶段推荐配置。

表 5.11 当前推荐配置

使用目标	推荐配置	理由
实时观看演示	detect_every_n=2，infer_workers=1，box_smooth=on，dynamic_roi=on	检测框更稳定，输出约 10 FPS，延迟约 110 ms
低延迟无目标巡检	detect_every_n=3，infer_workers=1，dynamic_roi=on	延迟较低，资源占用较小，适合一般实时查看
NPU 多线程实验	detect_every_n=1，infer_workers=2	能体现双 context 对吞吐的提升，适合作为硬件优化论证
后续优化方向	输入内存路径、RGA 预处理、量化输入匹配	当前瓶颈主要在 rknn_inputs_set 和 rknn_run

需要特别说明的是，INT8 量化和 RGA 硬件预处理目前不应写成已经完成的稳定优化结果。根据当前实验，FP RKNN 模型是稳定主路径，zero-copy 输入路径仍处于探索状态。论文中应将这些内容表述为后续优化方向，而不是已验证成果。

5.10 本章小结

本章围绕 RK3588 无人机检测系统进行了模型、固定视频、实时 RTSP、多 context 并行和阶段耗时等实验分析。训练结果表明当前单类无人机模型具备基本检测能力；公开视频固定输入验证表明恢复后的无人机 RKNN 模型能够在 RK3588 上稳定运行；实时 RTSP 实验说明多 context 可以提升吞吐，而检测间隔、动态 ROI 和框平滑更适合改善低延迟观看体验。

阶段耗时分析进一步指出，当前系统的主要瓶颈不是后处理和画框，而是 RKNN 输入设置和 NPU 执行阶段。zero-copy 实验表明，简单绑定输入内存不能直接解决性能问题，后续需要结合输入数据类型、模型量化方式和硬件预处理路径继续优化。

总体来看，本文已经形成了较完整的实验链路：模型训练、RKNN 转换、板端固定视频验证、实时 RTSP 演示、多 context 并行对比和瓶颈分析。下一章将基于前文设计、实现和实验结果，对全文工作进行总结，并讨论后续可改进方向。

总结与展望

本文围绕“基于嵌入式平台的目标检测系统研究”这一课题，完成了无人机目标检测模型在 RK3588 平台上的迁移、部署和系统级实验分析。本文以单类无人机检测模型为基础，完成模型导出、RKNN 转换、板端加载和输出张量解析，并在 C/C++ 程序中实现固定视频检测与实时 RTSP 检测两条运行路径。

在系统设计与实现方面，本文构建了面向实时检测的多线程处理流程，使视频输入、NPU 推理、结果绘制、推流输出和日志记录能够相互解耦。针对实时演示中出现的延迟、抖动和吞吐问题，系统实现了检测间隔控制、动态 ROI、轻量跟踪、检测框平滑和多 context 推理等策略。实验结果表明，双 context 在每帧检测模式下能够提升 NPU 推理吞吐；而在实际实时观看场景中，检测间隔配合框平滑更有利于降低端到端延迟并改善显示稳定性。

本文工作仍存在若干可继续完善之处。首先，当前主路径采用 FP RKNN 模型，INT8 量化尚未形成完整的精度和速度闭环验证，后续可基于更充分的校准数据开展量化实验。其次，当前预处理路径主要采用 OpenCV 实现，RGA 硬件预处理仍可作为后续优化方向。最后，外设 GPIO 闭环控制尚未作为本文已完成主线，后续可进一步接入报警、云台或其他执行机构，形成从目标感知到硬件响应的完整系统。

参考文献

[1] Redmon J, Farhadi A. YOLOv3: An Incremental Improvement[EB/OL]. arXiv:1804.02767, 2018[2026-04-27]. https://arxiv.org/abs/1804.02767.

[2] Bochkovskiy A, Wang C Y, Liao H Y M. YOLOv4: Optimal Speed and Accuracy of Object Detection[EB/OL]. arXiv:2004.10934, 2020[2026-04-27]. https://arxiv.org/abs/2004.10934.

[3] Wang A, Chen H, Liu L, et al. YOLOv10: Real-Time End-to-End Object Detection[EB/OL]. arXiv:2405.14458, 2024[2026-04-27]. https://arxiv.org/abs/2405.14458.

[4] Zhu X, Lyu S, Wang X, et al. TPH-YOLOv5: Improved YOLOv5 Based on Transformer Prediction Head for Object Detection on Drone-captured Scenarios[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops. 2021: 2778-2788.

[5] Huang B, Li J, Chen J, et al. Anti-UAV410: A Thermal Infrared Benchmark and Customized Scheme for Tracking Drones in the Wild[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2024, 46(5): 2852-2865.

[6] Tijtgat N, Van Ranst W, Goedeme T, et al. Embedded Real-Time Object Detection for a UAV Warning System[C]//Proceedings of the IEEE International Conference on Computer Vision Workshops. 2017: 2110-2118.

[7] Rockchip. RKNN Toolkit2 and RKNPU2 Documentation[EB/OL]. [2026-04-27]. https://github.com/airockchip/rknn-toolkit2.

[8] Ultralytics. Ultralytics YOLO Repository[EB/OL]. [2026-04-27]. https://github.com/ultralytics/ultralytics.

[9] Open Neural Network Exchange. ONNX Documentation[EB/OL]. [2026-04-27]. https://onnx.ai/onnx/.
[10] OpenCV Team. OpenCV Documentation[EB/OL]. [2026-04-27]. https://docs.opencv.org/.
[11] GStreamer Project. GStreamer Documentation[EB/OL]. [2026-04-27]. https://gstreamer.freedesktop.org/documentation/.

致谢

本课题的完成离不开指导教师在研究方向、系统设计和论文撰写方面给予的指导与帮助。在项目推进过程中，导师对 RK3588 嵌入式平台、NPU 多线程优化和系统级实验提出了明确要求，使本文能够从单纯模型部署进一步扩展到工程化实时系统设计与性能分析。

同时，感谢同学和实验环境提供者在模型、数据、开发板调试和测试场景方面给予的支持。本文所涉及的模型转换、板端部署、实时视频推流和公开数据验证经历了多轮调试，相关讨论和协助对系统最终跑通具有重要帮助。
最后，感谢家人和朋友在毕业设计期间给予的理解与支持。